COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CpE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S1
DATE PERFORMED:	9-10-2026	**DATE SUBMITTED:**	9-15-2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 [Sorting Algorithms PT2]

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

using namespace std;

const int SIZE = 100;

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";

        if ((i + 1) % 10 == 0)
        {
            cout << endl;
        }
    }
}

void copyArray(int source[], int destination[], int size)
{
    for (int i = 0; i < size; i++)
    {
        destination[i] = source[i];
    }
}
```

```
Original Unsorted Array:
433 764 537 728 977 342 854 120 520 688
583 432 655 454 921 658 16 437 404 207
641 718 942 498 436 938 715 682 652 776
68 740 978 634 157 265 321 391 564 84
151 61 871 111 757 148 780 525 569 603
711 563 354 797 64 483 467 876 395 937
468 66 612 530 621 979 210 617 420 807
883 244 585 758 904 406 904 5 875 991
39 952 734 441 409 272 190 340 754 415
544 36 980 818 507 273 129 527 398 792
```

```
int main()
{
    int original[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    srand(time(0));

    for (int i = 0; i < SIZE; i++)
    {
        original[i] = rand() % 1000 + 1;
    }

    copyArray(original, shellArray, SIZE);
    copyArray(original, mergeArray, SIZE);
    copyArray(original, quickArray, SIZE);

    cout << "Original Unsorted Array:" << endl;
    displayArray(original, SIZE);
```

Observation: The program successfully generated an array containing 100 random values. The values were initially unsorted and ranged from 5 to 991. The random arrangement made the array suitable for testing different sorting algorithms. The same set of values was used for Shell Sort, Merge Sort, and Quick Sort to make the comparison fair. This allowed us to observe how each sorting technique processes the same group of data.

Table 8-1. Array of Values for Sort Algorithm Testing

```
void shellSort(int arr[], int size)
{
    for (int interval = size / 2; interval > 0; interval /= 2)
    {
        for (int i = interval; i < size; i++)
        {
            int temp = arr[i];
            int j;

            for (j = i; j >= interval && arr[j - interval] > temp; j -= interval)
            {
                arr[j] = arr[j - interval];
            }

            arr[j] = temp;
        }
    }
}
```

```
shellSort(shellArray, SIZE);
```

```
Shell Sort:
5 16 36 39 61 64 66 68 84 111
120 129 148 151 157 190 207 210 244 265
272 273 321 340 342 354 391 395 398 404
406 409 415 420 432 433 436 437 441 454
467 468 483 498 507 520 525 527 530 537
544 563 564 569 583 585 603 612 617 621
634 641 652 655 658 682 688 711 715 718
728 734 740 754 757 758 764 776 780 792
797 807 818 854 871 875 876 883 904 904
921 937 938 942 952 977 978 979 980 991
```

Observation: Shell Sort successfully arranged all 100 values in ascending order. The algorithm gradually reduced the interval between elements to organize the values more efficiently. After the sorting process, the smallest value was 5 and the largest value was 991. The output shows that the algorithm correctly sorted all the elements without missing any values. This demonstrates that Shell Sort can effectively arrange a large number of randomly ordered elements.

Table 8-2. Shell Sort Technique

```
void merge(int arr[], int left, int middle, int right)
{
    int n1 = middle - left + 1;
    int n2 = right - middle;

    int L[n1];
    int R[n2];

    for (int i = 0; i < n1; i++)
    {
        L[i] = arr[left + i];
    }

    for (int j = 0; j < n2; j++)
    {
        R[j] = arr[middle + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }

        k++;
    }
```

```
Merge Sort:
5 16 36 39 61 64 66 68 84 111
120 129 148 151 157 190 207 210 244 265
272 273 321 340 342 354 391 395 398 404
406 409 415 420 432 433 436 437 441 454
467 468 483 498 507 520 525 527 530 537
544 563 564 569 583 585 603 612 617 621
634 641 652 655 658 682 688 711 715 718
728 734 740 754 757 758 764 776 780 792
797 807 818 854 871 875 876 883 904 904
921 937 938 942 952 977 978 979 980 991
```

```
    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int middle = (left + right) / 2;

        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);

        merge(arr, left, middle, right);
    }
}
```

```
mergeSort(mergeArray, 0, SIZE - 1);
```

Observation: Merge Sort successfully sorted all 100 random values in ascending order. The algorithm divided the array into smaller sections before merging them back together in the correct order. The final sorted array started with 5 and ended with 991. The result was the same as the output produced by Shell Sort and Quick Sort. This shows that Merge Sort was able to correctly process and organize the entire array.

Table 8-3. Merge Sort Algorithm

```
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivot = partition(arr, low, high);

        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

```
quickSort(quickArray, 0, SIZE - 1);
```

```
Quick Sort:
5 16 36 39 61 64 66 68 84 111
120 129 148 151 157 190 207 210 244 265
272 273 321 340 342 354 391 395 398 404
406 409 415 420 432 433 436 437 441 454
467 468 483 498 507 520 525 527 530 537
544 563 564 569 583 585 603 612 617 621
634 641 652 655 658 682 688 711 715 718
728 734 740 754 757 758 764 776 780 792
797 807 818 854 871 875 876 883 904 904
921 937 938 942 952 977 978 979 980 991
```

Observation: Quick Sort successfully arranged all 100 values from the smallest to the largest value. The algorithm used a pivot to divide the array into smaller sections and then sorted those sections recursively. The final output started at 5 and ended at 991, confirming that the values were sorted correctly. The sorted result was identical to the results of Shell Sort and Merge Sort. This shows that Quick Sort can efficiently organize a large set of randomly arranged values.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
int main()
{
    // PROBLEM 1
    std::cout << "PROBLEM 1" << std::endl;

    int arr1[] = {40, 20, 60, 10, 50, 30, 70};
    int size1 = 7;

    std::cout << "Original array: ";
    printArray(arr1, size1);

    // Partition the array
    int pivot = partition(arr1, 0, size1 - 1);

    std::cout << "After partition: ";
    printArray(arr1, size1);

    // using Insertion Sort
    insertionSort(arr1, 0, pivot - 1);
    insertionSort(arr1, pivot + 1, size1 - 1);

    std::cout << "After sorting sublists using Insertion Sort: ";
    printArray(arr1, size1);
```

```
PROBLEM 1
Original array: 40 20 60 10 50 30 70
After partition: 40 20 60 10 50 30 70
After sorting sublists using Insertion Sort: 10 20 30 40 50 60
```

Observation: The partition method divides the array into a left sublist and a right sublist based on the pivot. Instead of continuing with Quick Sort, Insertion Sort was used to arrange the two sublists. After both sublists were sorted, the complete array was successfully arranged in ascending order. This demonstrates that different sorting algorithms can work together with the partition method. Insertion Sort is especially useful when the sublists are relatively small.

Table 8-5. Problem 1

```
    // PROBLEM 2
    std::cout << std::endl;
    std::cout << "PROBLEM 2" << std::endl;

    int arr2[] = {
        4, 34, 29, 48, 53,
        87, 12, 30, 44, 25,
        93, 67, 43, 19, 74
    };

    int size2 = 15;

    std::cout << "Original array: ";
    printArray(arr2, size2);

    // Sort the array using Merge Sort
    mergeSort(arr2, 0, size2 - 1);

    std::cout << "Sorted array using Merge Sort: ";
    printArray(arr2, size2);

    return 0;
}
```

```
PROBLEM 2
Original array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
Sorted array using Merge Sort: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

Observation: Merge Sort successfully sorted the 15 elements in ascending order. It works by repeatedly dividing the array into smaller parts and then merging the sorted parts together. Merge Sort has an O(N log N) time complexity because the array is divided into about log N levels, while each level processes N elements. Quick Sort can also have O(N log N) time when the pivot creates balanced sublists. So the Merge Sort is a reliable choice when consistent sorting performance is important.

Table 8-6. Problem 2

Conclusion/learnings

I learned how different sorting algorithms work and how they can be applied to different types of data. I learned the processes of Shell Sort, Merge Sort, and Quick Sort, including how they arrange elements and how their performance differs. I also learned how Quick Sort uses partitioning to create left and right sublists and that these sublists can be sorted using other algorithms such as Insertion Sort. The supplementary problems also helped me understand why Merge Sort and Quick Sort can have O(N log N) time complexity and why choosing an appropriate sorting algorithm can improve performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **none**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **none**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **none**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.